Operating Systems Project

Phase 1 Report

CS-UH 3010 Operating Systems
Spring 2026

Group 6

Haroon Shafi (hs5312)
Saad Sifar (ss17886)

February 26, 2026

Contents

1 Architecture & Design **2**

1.1 High-Level Structure 2

1.2 Key Design Decisions 2

1.3 Data Structures and Algorithms 3

1.4 File Structure and Code Organization 3

2 Implementation Highlights **4**

2.1 Interactive Loop and Command Dispatch 4

2.2 Redirection Parsing 5

2.3 Single Command Execution 5

2.4 Pipeline Implementation 6

2.5 Error Handling and Edge Cases 7

3 Execution Instructions **7**

4 Testing **8**

4.1 Basic Command Execution 8

4.2 Input Redirection 9

4.3 Output Redirection 9

4.4 Error Redirection 9

4.5 Combined Redirection 10

4.6 Pipes 10

4.7 Multi-stage Pipes 11

4.8 Pipes with Redirection 11

4.9 Error Handling Scenarios 12

5 Challenges **13**

6 Division of Tasks **13**

7 References **14**

1 Architecture & Design

1.1 High-Level Structure

Our shell is organized as a **read-eval loop** in `main()`: it repeatedly prints a `$` prompt, reads a line of input with `fgets()`, and dispatches to one of two execution paths. If the line contains the pipe character `|`, the entire line is handled by `run_multi_piped_command()`; otherwise it is handled by `run_command()`. The built-in `exit` command is recognized in `main()` and terminates the process. All other commands are executed by forking a child process and using `execvp()`, with the parent waiting for the child so that the prompt reappears only after the command completes.

This **two-path design** keeps single-command execution simple (one `fork()`, one `execvp()`, one `wait()`) while pipelines get dedicated logic. The pipeline handler splits the line by `|`, creates one pipe per adjacent pair of commands, forks one child per command, connects each child's standard input/output to the appropriate pipe ends, and then applies the same redirection and execution logic used for single commands. As a result, the system has a clear separation between commands without pipes and commands with one or more pipes, while reusing shared parsing and execution components.

1.2 Key Design Decisions

Redirections before tokenization. Redirections are parsed first via `parse_redirections()`, which locates `2>`, `>`, and `<` in the command string, null-terminates the string at those points, and sets pointers to the filenames that follow. This modifies the command string in place so that the remaining text contains only the program name and its arguments. Tokenization with `strtok()` is then applied to this trimmed string. Parsing redirections first avoids treating redirection operators or their targets as command arguments and ensures a consistent flow for both single commands and pipeline stages.

Shared redirection parsing. Both `run_command()` (single command) and `run_multi_piped_command()` (each stage of a pipeline) use the same `parse_redirections()` logic and, in the pipeline case, the helper `execute_command_with_redirections()`. This guarantees that every command supports `<`, `>`, and `2>` in the same way, and centralizes error handling (e.g., missing filename after `>`) for consistency.

Pipeline implementation. For a line with n commands (e.g., `cmd1 | cmd2 | cmd3`), the code creates $n - 1$ pipes, then forks n children. Child i reads from `pipes[i-1][0]` (unless it is the first command) and writes to `pipes[i][1]` (unless it is the last). All pipe file descriptors are closed in each child before `exec` to

prevent blocking behavior. The parent closes all pipe ends and waits for all children with `waitpid()`, ensuring that the shell prompts again only after the entire pipeline has completed.

1.3 Data Structures and Algorithms

- **Command representation:** The raw command is stored in a fixed-size buffer (`char command[256]`). After redirection parsing, the same buffer holds only the executable and its arguments. Pointers into this buffer are passed to `execvp()` as a NULL-terminated array (`char *args[10]`). The fixed size simplifies memory management and is sufficient for the required use cases.

- **Tokenization:** `strtok()` is used with space as the delimiter to split the command into the `args[]` array, matching standard shell behavior where arguments are space-separated.

- **Redirection detection:** `strstr()` is used for detecting `"2>"` so that error redirection is recognized before a lone >. For > and <, `strchr()` is used. The order of checks in `parse_redirections()` (error, then output, then input) ensures that 2> is not misinterpreted as standard output redirection.

- **Pipeline splitting:** The number of commands is computed by counting `|` characters and adding one. The line is then split with `strtok(command, "|")`. Pipe file descriptors are stored in a two-dimensional array (`pipes[num_pipes][2]`) and child process IDs in `pids[num_commands]`, using C99 variable-length arrays to support an arbitrary number of pipeline stages.

1.4 File Structure and Code Organization

Our shell is contained in a single source file, **`myshell.c`**, which compiles to the `myshell` executable. The file is organized into the following logical functions:

- **`main()`** — Implements the read-eval loop, prompt display, input reading, newline stripping, and dispatch for `exit`, pipeline, or single command.

- **`parse_redirections()`** — Parses 2>, >, and < along with their targets; returns 0 on success and -1 on error, printing required error messages for missing filenames.

- **`run_command()`** — Handles single-command execution: calls `parse_redirections()`, tokenizes into `args[]`, forks, applies `dup2()` and `open()` for redirections in the child, executes with `execvp()`, and waits in the parent.

- **`execute_command_with_redirections()`** — Used by pipeline stages to apply redirections, tokenize the command, and execute via `execvp()`, avoiding duplication of logic.
- **`run_multi_piped_command()`** — Splits the line by `|`, validates structure, creates pipes, forks one child per command, connects appropriate pipe ends, applies redirection parsing and execution logic, closes all pipe descriptors in the parent, and waits for all children.

2 Implementation Highlights

Now we will go through the core implementation of `myshell`, focusing on command parsing, execution, pipelines, and error handling. All functionality is implemented in `myshell.c` using standard Linux system calls such as `fork()`, `execvp()`, `wait()`/`waitpid()`, `pipe()`, `dup2()`, and `open()`.

2.1 Interactive Loop and Command Dispatch

The shell runs as a read-eval loop in `main()`, which prints the prompt, reads input using `fgets()`, removes the trailing newline, and dispatches execution based on the command:

```
while (1) {
    printf("$ ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0) {
        exit(0);
    } else if (strchr(command, '|') != NULL) {
        run_multi_piped_command(command);
    } else {
        run_command(command);
    }
}
```

We designed this to cleanly separate built-in handling, single commands, and pipelines.

2.2 Redirection Parsing

Redirections (<, >, 2>) are handled by `parse_redirections()`, which modifies the command string in place. It detects operators, splits the string using `'\0'`, trims whitespace, and validates that filenames are provided.

```
char *err_redir = strstr(command, "2>");
if (err_redir != NULL) {
    *err_redir = '\0';
    err_redir += 2;
    while (*err_redir == ' ') err_redir++;
    if (strlen(err_redir) == 0) {
        fprintf(stderr, "Error output file not specified.\n");
        return -1;
    }
    *errfile = err_redir;
}
```

Parsing is performed before tokenization to prevent redirection operators from being treated as normal arguments. The function returns -1 if syntax is invalid, preventing execution.

2.3 Single Command Execution

Single commands are executed in `run_command()`. The process is:

1. Parse redirections.
2. Tokenize arguments using `strtok()`.
3. Fork a child process.
4. Apply any redirections using `open()` and `dup2()`.
5. Execute using `execvp()`.
6. Parent waits using `wait()`.

Core execution logic:

```
pid_t pid = fork();
```

```
if (pid == 0) {
    if (outfile != NULL) {
        int fd = open(outfile, O_WRONLY|O_CREAT|O_TRUNC, 0644);
        dup2(fd, 1);
        close(fd);
    }

    execvp(args[0], args);
    fprintf(stderr, "Command not found: %s\n", args[0]);
    exit(1);
} else {
    wait(NULL);
}
```

Each command runs in a child process, satisfying the requirement that the shell waits for completion before printing the next prompt.

2.4 Pipeline Implementation

Pipelines are implemented in `run_multi_piped_command()`. The algorithm:

- Count commands by counting `|`.
- Split the input into stages.
- Create $N - 1$ pipes.
- Fork N children.

Pipe wiring in each child:

```
if (i > 0)
    dup2(pipes[i-1][0], 0);

if (i < num_commands - 1)
    dup2(pipes[i][1], 1);

for (int j = 0; j < num_pipes; j++) {
    close(pipes[j][0]);
    close(pipes[j][1]);
}
```

Each stage then parses its own redirections and executes via the helper:

```
execute_command_with_redirections(
    commands[i], infile, outfile, errfile);
```

The parent closes all pipe file descriptors and waits for all children using `waitpid()`, ensuring the shell prompt appears only after the entire pipeline completes.

2.5 Error Handling and Edge Cases

Our shell implementation explicitly handles common syntax and runtime errors:

- **Missing redirection filenames:** Detected in `parse_redirections()`, e.g., `command >`. Execution is aborted.
- **Empty pipeline stages:** Inputs like `cmd1 || cmd2` produce "Empty command between pipes."
- **Trailing pipe:** `cmd1 |` produces "Command missing after pipe."
- **File open failures:** If `open()` fails, a descriptive error is printed and the child exits.
- **Invalid commands:** If `execvp()` fails, the shell prints "Command not found: [cmd]" and exits with a non-zero status.

Our checks ensure the shell fails safely without crashing and provides clear feedback while maintaining correct process behavior.

3 Execution Instructions

1. Download and unzip the project files.
2. Open a terminal in the project directory.
3. Compile the program: `gcc myshell.c -o myshell`
4. Run the shell: `./myshell`
5. Alternatively, using the make file just run `make` and then ./myshell
6. To clean the make, run `make clean`

4 Testing

The following test cases were executed to verify correctness and robustness.

4.1 Basic Command Execution

- `ls // Expect: directory listing on screen`

```
$ ls
a.out                   myshell                 so
err.log                 myshell.c               sorted.txt
fruits.txt              out1.txt                test_combinations.sh
hello                   output.txt              test.txt
hello.c                 overwrite.txt           unsorted.txt
input.txt               README.md
```

- `ls -l // example: input w/ arguments - list info about files`

```
$ ls -l
total 336
-rwxr-xr-x@ 1 saadsifar  staff  34760 Feb 26 20:54 a.out
-rw-r--r--@ 1 saadsifar  staff      0 Feb 23 22:34 err.log
-rw-r--r--@ 1 saadsifar  staff     27 Feb 23 22:32 fruits.txt
-rwxr-xr-x@ 1 saadsifar  staff  33432 Feb 23 22:17 hello
-rw-r--r--@ 1 saadsifar  staff    148 Feb 23 22:17 hello.c
-rw-r--r--@ 1 saadsifar  staff    130 Feb 23 22:29 input.txt
-rwxr-xr-x@ 1 saadsifar  staff  34760 Feb 26 20:54 myshell
-rw-r--r--@ 1 saadsifar  staff  10371 Feb 23 22:19 myshell.c
-rw-r--r--@ 1 saadsifar  staff    182 Feb 23 22:33 out1.txt
-rw-r--r--@ 1 saadsifar  staff      9 Feb 23 22:17 output.txt
-rw-r--r--@ 1 saadsifar  staff      7 Feb 23 22:30 overwrite.txt
-rw-r--r--@ 1 saadsifar  staff   1602 Feb 23 22:17 README.md
-rw-r--r--@ 1 saadsifar  staff     22 Feb 23 22:17 so
```

- `exit // expect shell terminates immediately`

```
$ exit
saadsifar@Saads-MacBook-Air os-project %
```

- `./myshell // running other programs (from another shell)`

```
saadsifar@Saads-MacBook-Air os-project % ./myshell
$ ./hello
Hello from custom program!
This program was executed by myshell.
$
```

4.2 Input Redirection

- `cat < input.txt // Expect:  prints hello world`

```
$ echo "hello world" > input.txt
$ cat < input.txt
"hello world"
$
```

- `wc -l < lines.txt // Expect:  number of lines in file (1)`

```
$ echo -e "Hello World" > lines.txt
$ wc -l < lines.txt
       1
$
```

4.3 Output Redirection

- `ls > out.txt // expect contents of ls in out.txt`

```
$ ls > out.txt
$ cat out.txt
a.out
err.log
fruits.txt
hello
hello.c
input.txt
lines.txt
myshell
myshell.c
out.txt
out1.txt
output.txt
overwrite.txt
README.md
```

4.4 Error Redirection

- `ls non_existent 2> err.log // Expect error in err.log`

```
$ ls non_existent 2> err.log
$ cat err.log
ls: non_existent: No such file or directory
$
```

4.5 Combined Redirection

- `tr a-z A-Z < input.txt > upper.txt // transforms contents of input.txt to upper case and saves to upper.txt`

```
$ cat input.txt
"hello world"
$ tr a-z A-Z < input.txt > upper.txt
$ cat upper.txt
"HELLO WORLD"
$
```

4.6 Pipes

- `ls | wc -l // expect number of files in current directory`

```
$ ls | wc -l
     20
$
```

- `echo "hello" | grep "hello" // expect returns hello line`

```
$ echo -e "hello" | grep "hello"
-e "hello"
$
```

4.7 Multi-stage Pipes

- `echo test.txt | grep b | wc -c // expect returns number of characters in line containing b (including newline character)`

```
$ echo -e "a\nbb\nccc" > test.txt
$ cat test.txt | grep b | wc -c
       3
$
```

4.8 Pipes with Redirection

Examples tested:

- `cat < fruits.txt | wc -l // expect count of lines in fruits.txt`

```
$ cat fruits.txt
apple
banana
cherry
$ cat < fruits.txt | wc -l
       3
$
```

- `ls | wc -l > count.txt // expect count of files in count.txt`

```
$ ls | wc -l > count.txt
$ cat count.txt
      21
$
```

- cat < fruits.txt | grep a | wc -l > a_count.txt // expect number of lines where a appears

```
$ cat < fruits.txt | grep a | wc -l > a_count.txt
$ cat fruits.txt
apple
banana
cherry
$ cat a_count.txt
       2
```

4.9 Error Handling Scenarios

- cat < // expect: input file not specified
- ls > // expect: output file not specified
- ls 2> // expect: error output file not specified
- ls | // expect: command missing after pipe
- ls || wc // expect: command missing after pipe
- invalid_command // expect: command not found: invalid_command
- ls | invalid_command | wc // expect: command not found: invalid_comma
- < input.txt | wc > // expect: output file not specified.
- cat < non_existent_file.txt // expect cannot open input file: file not found

```
$ cat <
Input file not specified.
$ ls >
Output file not specified.
$ ls 2>
Error output file not specified.
$ ls |
Command missing after pipe.
$ ls || wc
Command missing after pipe.
$ invalid_command
Command not found: invalid_command
$ ls | invalid_command | wc
Command not found: invalid_command
       0       0       0
$ < input.txt | wc >
Output file not specified.
$ cat < non_existent_file.txt
Error: Cannot open input file 'non_existent_file.txt': File not found.
$
```

5 Challenges

The primary challenges included:

- Correctly parsing complex command structures
- Managing file descriptors safely
- Ensuring proper closure of unused pipe ends
- Preventing segmentation faults from malformed input

These were resolved through incremental testing, debugging with print statements, and careful validation of user input before execution.

6 Division of Tasks

Saad Sifar:

- Implemented command execution with and without arguments
- Implemented input and output redirection

Haroon Shafi:

- Implemented pipes (single and multi-stage)
- Implemented compound command combinations
- Implemented advanced error handling

Both of us:

- Conducted testing
- Reviewed each other's code
- Collaborated on writing this report

7 References

- C standard documentation
- Course lecture notes and lab exercises
- Large Language Models (e.g., ChatGPT, Google Gemini) for conceptual understanding of parsing, system calls, file I/O, and aiding in the writing and formatting of this report